Projet SIO-Shoes : site « e-commerce » avec Symfony

Partie 15 : total à payer

1. Ajout des frais de port et du total à payer dans la commande.

Une fois la commande passée, le prix à payer (total du prix des produits + frais de port) ne dois pas changer (alors que les prix des produits évoluent et que les frais de port peuvent également changer).

Commencez par modifier l'entité « Order » avec la commande suivante :
`symfony console make:entity Order`

Ajoutez les champ « shippingPrice » et totalPrice (tous les deux de type « decimal ») :

```
PS C:\Symfony\e-commerce> symfony console make:entity Order
 Your entity already exists! So let's add some new fields!

 New property name (press <return> to stop adding fields):
 > shippingPrice

 Field type (enter ? to see all types) [string]:
 > decimal

 Precision (total number of digits stored: 100.00 would be 5) [10]:
 >

 Scale (number of decimals to store: 100.00 would be 2) [0]:
 > 2

 Can this field be null in the database (nullable) (yes/no) [no]:
 >

 updated: src/Entity/Order.php

 Add another property? Enter the property name (or press <return> to stop adding fields):
 > totalPrice

 Field type (enter ? to see all types) [string]:
 > decimal

 Precision (total number of digits stored: 100.00 would be 5) [10]:
 >

 Scale (number of decimals to store: 100.00 would be 2) [0]:
 > 2

 Can this field be null in the database (nullable) (yes/no) [no]:
 >

 updated: src/Entity/Order.php

 Add another property? Enter the property name (or press <return> to stop adding fields):
 >


  Success!

 Next: When you're ready, create a migration with symfony.exe console make:migration
```

2. **Préparez et exécutez la migration pour mettre à jour la BDD :**

```
symfony console make:migration
symfony console doctrine:migrations:migrate
```

3. **Modifiez le fichier « src\Form\OrderTypeForm.php» pour ajouter le champ « shippingPrice » en utilisant la classe « hidden » (caché).**

   En effet, l'utilisateur ne doit pas pouvoir saisir le montant des frais de port, calculés automatiquement en fonction de la ville de la ville de livraison choisie :

```php
$builder
            ->add('firstName')
            ->add('lastName')
            ->add('phoneNumber')
            ->add('adress')
            //->add('createdAt', null, [
            //    'widget' => 'single_text',
            //])
            ->add('city', EntityType::class, [
                'class' => City::class,
                'choice_label' => function($city) {
                    return $city->getZipCode() . " - " . $city->getName();
                },
                'query_builder' => function (EntityRepository $er) {
                    return $er->createQueryBuilder('c')
                      ->orderBy('c.zipCode', 'ASC')
                      ->addOrderBy('c.name', 'ASC');
                },
            ])
             ->add('shippingPrice',HiddenType::class)
        ;
```

4. **Mise à jour des frais de port en fonction de la ville sélectionnée dans le formulaire de commande.**

   Ouvrez le fichier « src\Controller\OrderController.php » et ajoutez la bibliothèque « JsonResponse » : `use Symfony\Component\HttpFoundation\JsonResponse;`

   Créez ensuite une nouvelle route « /get_shipping_cost » qui va nous permettre de récupérer le montant des frais de ports puis de les renvoyer au formulaire au format JSON :

```php
#[Route('/get_shipping_cost', name: 'get_shipping_cost', methods: ['POST'])]
public function getShippingCost(Request $request, CityRepository $cityRepository):
JsonResponse
{
        $id = $request->request->get('city');
        $city = $cityRepository->find($id);
        $cost = $city->getShippingCost() ?? 10.00; //10€ par défaut
```

```
        return new JsonResponse(['shippingCost' => $cost]);
    }
```

5. **Modifiez la vue « templates\order\index.html.twig » pour modifier dynamiquement l'affichage des frais de port en appelant la route « get_shipping_cost » lors du choix d'une ville dans le formulaire.**

   On utilisera Javascript pour détecter la sélection de la ville et appliquer les changements :

```
{% extends 'base.html.twig' %}

{% block title %}Commande{% endblock %}

{% block body %}
   <div class = "container">
        <br>
        <h1>Commande</h1>
        <br>
        <div>
            <h4>Montant de la commande : {{ total|number_format(2, '.', ',') }} €</h4>
            <strong>
                Frais de livraison : <span id="shipping-cost"></span>
            </strong>
            <h4>
                Montant total : <span id="total-cost"></span>
            </h4>
        </div>

        {{ include('order/_form.html.twig') }}
        <br>
        <a class="btn btn-dark" href="{{ path('app_cart') }}">Retour</a>
    </div>

    <script>
        document.addEventListener('DOMContentLoaded', function () {
            const citySelect = document.getElementById('order_type_form_city');
            const shippingCostDisplay = document.getElementById('shipping-cost');
            const shippingCostForm = document.getElementById('order_type_form_shippingPrice');
            const totalCostDisplay = document.getElementById('total-cost');

            function updateShippingCost(city) {
                fetch('{{ path('get_shipping_cost') }}', {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/x-www-form-urlencoded',
                        'X-Requested-With': 'XMLHttpRequest'
                    },
```

```
                    body: new URLSearchParams({city: city})
                })
                .then(response => response.json())
                .then(data => {
                    shippingCostDisplay.textContent = data.shippingCost + ' €';
                    shippingCostForm.value = data.shippingCost + ' €';
                    totalCostDisplay.textContent = ({{ total }} +
                    parseFloat(data.shippingCost)).toFixed(2) + ' €';
                })
                .catch(error => {
                    console.error('Erreur lors de la récupération des frais de
                    livraison :', error);
                });
            }

            if (citySelect) {
                citySelect.addEventListener('change', function () {
                 updateShippingCost(citySelect.options[citySelect.selectedIndex].value);
                });

                // Initialiser le coût si une ville est présélectionnée
                if (citySelect.value) {
                    updateShippingCost(citySelect.options[citySelect.selectedIndex].value);
                }
            }
        });
    </script>
{% endblock %}
```

Remarque : l'intitulé des « id » du formulaire est généré automatiquement par symfony et peut donc varier d'une application à une autre. Vérifiez le nom des id de votre formulaire en utilisant le mode développeur de votre navigateur (F12 pour Chrome ou Edge).

6. **Modifiez la route « /order » du contrôleur « src\Controller\OrderController.php » pour ajouter l'enregistrement des frais de port et du montant total de la commande :**

```
if ($form->isSubmitted() && $form->isValid())
{
    $order->setCreatedAt(new \DateTimeImmutable());
    $order->setTotalPrice($total +  $order->getShippingPrice());

    $entityManager->persist($order);
    $entityManager->flush();

    return $this->redirectToRoute('app_home', [], Response::HTTP_SEE_OTHER);
}
```

Enregistrez une commande et vérifiez le résultat dans la BDD :



7. Ajout des produits à la commande.

Nous allons créer l'entité « OrderProducts » qui contiendra l'id de la commande, l'id du produit, la quantité et le prix facturé.

Exécutez la commande `symfony console make:entity OrderProducts`

```
PS C:\Symfony\e-commerce> symfony console make:entity OrderProducts
 created: src/Entity/OrderProducts.php
 created: src/Repository/OrderProductsRepository.php

 Entity generated! Now let's add some fields!
 You can always add more fields later manually or by re-running this command.

 New property name (press <return> to stop adding fields):
 > _order

 Field type (enter ? to see all types) [string]:
 > ManyToOne

 What class should this entity be related to?:
 > Order

 Is the OrderProducts._order property allowed to be null (nullable)? (yes/no) [yes]:
 > no

 Do you want to add a new property to Order so that you can access/update OrderProducts objects from it - e.g.
 $order->getOrderProducts()? (yes/no) [yes]:
 >

 A new property will also be added to the Order class so that you can access the related OrderProducts objects
 from it.

 New field name inside Order [orderProducts]:
 >

 Do you want to activate orphanRemoval on your relationship?
 A OrderProducts is "orphaned" when it is removed from its related Order.
 e.g. $order->removeOrderProducts($orderProducts)

 NOTE: If a OrderProducts may *change* from one Order to another, answer "no".

 Do you want to automatically delete orphaned App\Entity\OrderProducts objects (orphanRemoval)? (yes/no) [no]:
 >

 updated: src/Entity/OrderProducts.php
 updated: src/Entity/Order.php
```

```
Add another property? Enter the property name (or press <return> to stop adding fields):
> product

Field type (enter ? to see all types) [string]:
> ManyToOne

What class should this entity be related to?:
> Product

Is the OrderProducts.product property allowed to be null (nullable)? (yes/no) [yes]:
> no

Do you want to add a new property to Product so that you can access/update OrderProducts objects from it - e.
g. $product->getOrderProducts()? (yes/no) [yes]:
>

A new property will also be added to the Product class so that you can access the related OrderProducts objec
ts from it.

New field name inside Product [orderProducts]:
>

Do you want to activate orphanRemoval on your relationship?
A OrderProducts is "orphaned" when it is removed from its related Product.
e.g. $product->removeOrderProducts($orderProducts)

NOTE: If a OrderProducts may *change* from one Product to another, answer "no".

Do you want to automatically delete orphaned App\Entity\OrderProducts objects (orphanRemoval)? (yes/no) [no]:
>

updated: src/Entity/OrderProducts.php
updated: src/Entity/Product.php
```

```
Add another property? Enter the property name (or press <return> to stop adding fields):
> qte

Field type (enter ? to see all types) [string]:
> integer

Can this field be null in the database (nullable) (yes/no) [no]:
>

updated: src/Entity/OrderProducts.php

Add another property? Enter the property name (or press <return> to stop adding fields):
> price

Field type (enter ? to see all types) [string]:
> decimal

Precision (total number of digits stored: 100.00 would be 5) [10]:
>

Scale (number of decimals to store: 100.00 would be 2) [0]:
> 2

Can this field be null in the database (nullable) (yes/no) [no]:
>

updated: src/Entity/OrderProducts.php

Add another property? Enter the property name (or press <return> to stop adding fields):
>


 Success!

Next: When you're ready, create a migration with symfony.exe console make:migration
```

8. **Préparez et exécutez la migration pour mettre à jour la BDD :**

```
symfony console make:migration
symfony console doctrine:migrations:migrate
```

9. Affichage de la confirmation de la confirmation de la commande.

Commencez par créer une Commnouvelle route « order-message » dans le contrôleur « src\Controller\OrderController.php » :

```
#[Route('/order-message', name: 'order_message')]
public function orderMessage(CategoryRepository $categoryRepository): Response
{
        return $this->render('order/order-message.html.twig', [
            'categories' => $categoryRepository->findAll(),
        ]);
}
```

Créez ensuite un nouveau fichier « templates\order\order-message.html.twig » pour la vue :

```
{% extends 'base.html.twig' %}

{% block title %}Commande{% endblock %}

{% block body %}
    <div class = "container">
        <h2>Commande enregistrée</h2>
    </div>
{% endblock %}
```

10. Modifiez la route « order » du contrôleur « src\Controller\OrderController.php » pour ajouter l'enregistrement des produits avec leur quantité et leur prix dans la table « order_products » :

```
if ($form->isSubmitted() && $form->isValid())
{
if(!empty($cart))
{
            $order->setCreatedAt(new \DateTimeImmutable());
            $order->setTotalPrice($total +  $order->getShippingPrice());

            $entityManager->persist($order);
            $entityManager->flush();

            foreach ($cart as $id => $quantity) {
                   $product = $productRepository->find($id);

                   if ($product) {
                        $orderProduct = new OrderProducts();
                        $orderProduct->setOrder($order);
                        $orderProduct->setProduct($product);
                        $orderProduct->setQte($quantity);
                        $orderProduct->setPrice($product->getPrice());
```

```
                    $entityManager->persist($orderProduct);
                    $entityManager->flush();
                }
            }
            $session->remove('cart'); //On vide le panier
            return $this->redirectToRoute('order_message', [], Response::HTTP_SEE_OTHER);
        }
        else
        {
            //message commande annulée
             $this->addFlash('danger','Impossible de commander, le panier est vide !');
             return $this->redirectToRoute('app_cart', [], Response::HTTP_SEE_OTHER);
        }
}
…
```